

Section 1:

1.1 Purpose

This document presents a high-level overview of the Nutrify system architecture, outlining key design considerations for the overall system and its various subsystems.

This document is intended for internal stakeholders, including but not limited to, project managers, developers, domain experts, and Nutrify team members / investors. Nutrify Deliverable 1 should be reviewed beforehand. While technical knowledge is not required, it may help in understanding the document's contents more effectively.

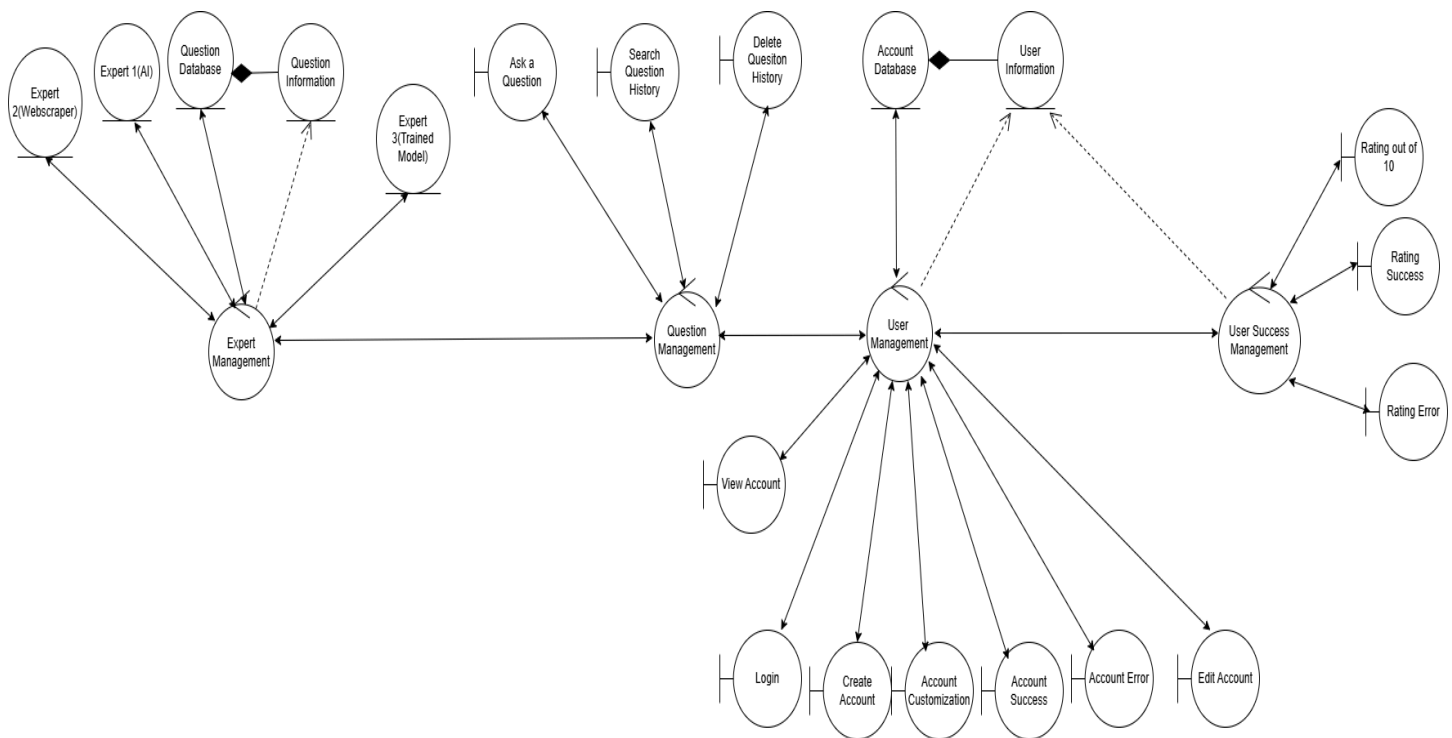
1.2.

The system follows a blackboard architecture, where multiple specialized subsystems collaborate by reading from and writing to a shared data structure. This architecture enables dynamic problem-solving, allowing independent agents to contribute partial solutions until a final decision is reached. The blackboard serves as a central data store, with a controller managing data-driven interactions between subsystems. This design aligns with Nutrify's core functionality, where three experts (agents) provide possible solutions to the user's question. Once all agents contribute their answers, the controller evaluates and selects the best one.

1.3.

Section 2 shows an Analysis Class Diagram for Nutrify. Section 3 explains why blackboard will be used as the system's architectural design, while also covering why alternative architectures did not fit Nutrify's functionality and requirements. Finally, Section 4 presents Class Responsibility Collaboration (CRC) Cards to break down classes, their responsibilities, and how they interact.

Section 2:



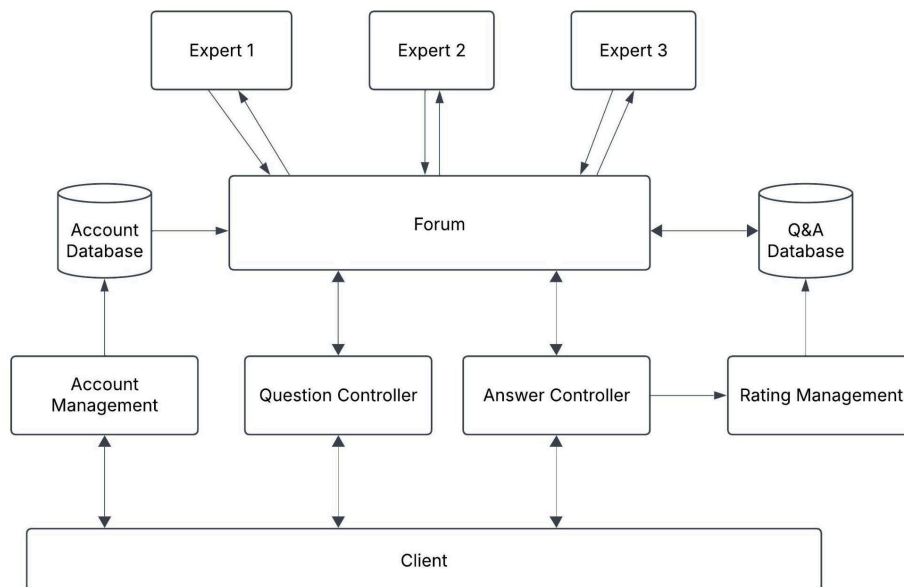
Section 3:

3.1 System Architecture

The Nutrify app follows a Blackboard Architecture pattern. In this architecture, a central hub, the forum, serves as the main point of interaction where questions and answers can be managed and shared. The system is composed of three knowledge agents (Expert 1, Expert 2 and Expert3) that each provide their own specialized insights and contribute to generating the best possible answer. These knowledge agents work independently but contribute their responses to the central forum, which acts as the blackboard where all the intermediate solutions are gathered and processed.

In addition, the system includes two controllers: the Answer Controller and the Question Controller, which are responsible for managing interactions with the Forum and ensuring smooth data flow. Two databases, the Q&A Database and Account Database store the relevant data. The Account Database is also linked to the Account Management module which handles user account information and authentication. A Rating Management module is integrated with the Answer Controller and the Q&A Database to allow users to rate responses, improving future interactions. Both the Account Management module and the two controllers are linked to the Client interface, which serves as the user-facing component of the system.

The Blackboard Architecture was chosen due to its ability to handle complex problem-solving scenarios where multiple specialized agents collaborate to generate solutions. In this system, the central forum acts as the blackboard, serving as a shared repository where questions and answers are posted and managed. The three knowledge agents independently contribute their expertise to the blackboard, allowing for a collaborative approach to determining the best answer. This architecture supports modularity and scalability, as new experts can be added or removed without disrupting the existing system, making it adaptable to future enhancements. The separation of concerns between the Question Controller, Answer Controller, and Rating Management ensures efficient handling of user interactions and feedback. Additionally, the architecture supports concurrent access and interaction, which is essential for the app as multiple users can ask questions and receive answers simultaneously.



For the Data-Centered architecture styles, we ended up not going with the Repository style due to its lack of dynamic collaboration between agents/modules and central data stores' passiveness. The system's core functionality relies on multiple expert agents providing their own solutions to a problem, sometimes if applicable, based off each other's solutions. This is not direct communication between the agents, which in fact both Blackboard and Repository don't allow, but it's indirect communication that is not supported in the Repository style. Additionally, after all potential solutions are given, the central data store chooses the best one. This also is not supported in the Repository style since the central data store (repository) is passive and does not control flow of operations. It is true that there can be a module that is responsible for controlling operational flow, but it is extra work, and this functionality is better supported in the Blackboard architecture anyways, where the central data store (blackboard) is active.

We considered the Batch Sequential architecture style but chose not to due to its inability to support real time interaction. The system is interactive with users in real time, where they expect immediate, context-aware responses. Batch-processing in the Batch Sequential style would introduce unwanted delays in the response due to its sequential nature of processing data in batches. Due to that, it provides low throughput and thus is not applicable for a high throughput, live systems like Nutrify.

We considered the Pipe & Filter architecture style but chose not to due to its restricted compatibility with elementary data types. This architecture divides the system into filters (processing elements) connected by pipes (data channels). The data that flows between these pipes is limited to binary or character streams, and since the input data from our system will not be in one of those forms, it makes the architecture incompatible.

We eliminated the Process-Control architecture style due to its mismatch with the application domain. This architecture is used for real-time control of physical processes or embedded systems. There are no physical control concerns related to any of the system's components or functionalities, thus making the architecture style not relevant for Nutrify.

3.2 Subsystems

Our main system has been divided into 4 main subsystems: Answer Management, Questions Management, Account Management, and User Success Management.

The Answer Management subsystem is mostly in charge of handling the responses of the experts and storing questions in a Question Database. Question information is given to the experts to individually solve the question. The answers are then given back to the Answer Management controller to select the best answer to return to the user. It communicates with the Question Management controller to receive user questions and adheres to the Blackboard software architecture.

The Questions Management subsystem is responsible for managing user questions, such as asking a question, searching question history, and deleting question history. It communicates with the Answer Management subsystem to provide a user question and receive the expert answer and the Account Management subsystem to associate the question with a particular user.

The Account Management subsystem manages the accounts of users, letting users view, create, and edit their account, login or customize their account. It communicates with the User Success management subsystem to retrieve account and question information.

The User Success Management subsystem allows the user to rate each answer from a scale from 1 to 10. This information provides important feedback for the system to developers, which can help give better answers to users in future updates.

Section 4:

Class Name: Question Information (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Controller Knows Question Database	Expert Management Controller

Class Name: Ask a Question (Boundary)	
Responsibility:	Collaborators:
Knows Question Management Controller Handles asking questions event	Question Management Controller

Class Name: View Account (Boundary)	
Responsibility:	Collaborators:
Knows Account Management Controller Handles click-event of "View Account" button Displays user information (email, username)	Account Management Controller

Class Name: Login (Boundary)	
Responsibility:	Collaborators:
Knows Account Management Controller Handles click-event of "Login" button	Account Management Controller

Class Name: Question Error (Boundary)	
Responsibility:	Collaborators:

Knows Question Management Controller Handles event of "Ask Question Again" button	Question Management Controller
--	--------------------------------

Class Name: Expert Answer Error (Boundary)	
Responsibility:	Collaborators:
Knows Expert Management Controller Handles event of re-generating answer	Expert Management Controller

Yash

Class Name: Expert 1 - AI (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Controller Handles Answering Questions Using AI	Expert Management Controller

Class Name: Expert 2 - Webscraper (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Controller Handles Answering Questions Using Webscraper	Expert Management Controller

Class Name: Expert 3 - Trained Model (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Controller Handles Answering Questions Using Trained Model	Expert Management Controller

Class Name: Delete Question History (Boundary)	
Responsibility:	Collaborators:
Knows Question Management Controller	Question Management Controller

Handles Click Event of “Delete Question History” Button	
---	--

Class Name: Account Database (Entity)	
Responsibility:	Collaborators:
Knows User Management Controller Composed of User Information Handles Account Storage	User Management Controller User Information

Class Name: Edit Account (Boundary)	
Responsibility:	Collaborators:
Knows User Management Controller Handles Click Event of “Edit Account” Button	User Management Controller

Class Name: Question Database (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Composed of Question Information Handles question storage	Expert Management Question Information

Class Name: Search Question History (Boundary)	
Responsibility:	Collaborators:
Knows Question Management Handles question history access	Question Management

Class Name: User Information (Entity)	
Responsibility:	Collaborators:
Knows Account Database Knows User Management Knows User Success Management Knows Username	Account Database User Management User Success Management

Knows Password Knows Date-of-birth Knows Dietary Restrictions	
---	--

Class Name: Create Account (Boundary)	
Responsibility:	Collaborators:
Knows User Management Knows Username Knows Password Handles click-event "Create Account"	User Management

Class Name: Account Customization Boundary	
Responsibility:	Collaborators:
Knows User Management Knows Username Knows Password Knows Gender Knows Date-of-birth Knows Dietary Restrictions Handles click-event of "Apply" button	User Management

Class Name: Account Success (Boundary)	
Responsibility:	Collaborators:
Knows User Management Handles unsuccessful events of saving/ creating account	User Management

Class Name: Account Error (Boundary)	
Responsibility:	Collaborators:
Knows User Management Handles time-out event event Handles Saving error event Handles Creating account error event	User Management

Class Name: Rating out of 10 (Boundary)	
Responsibility:	Collaborators:
Knows User Success Management	User Success Management

Class Name: Rating Success (Boundary)	
Responsibility:	Collaborators:
Knows User Success Management Handles unsuccessful events of rating	User Success Management

Class Name: Rating Error (Boundary)	
Responsibility:	Collaborators:
Knows User Success Management Handles event of "Retry rating" button	User Success Management

Class Name: Question Database (Entity)	
Responsibility:	Collaborators:
Knows Expert Management Composed of Question Information Handles question storage	Expert Management Question Information

Class Name: Search Question History (Boundary)	
Responsibility:	Collaborators:
Knows Question Management Handles question history access	Question Management

Class Name: User Information (Entity)	
---------------------------------------	--

Responsibility:	Collaborators:
Knows Account Database Knows User Management Knows User Success Management Knows Username Knows Password Knows Date-of-birth Knows Dietary Restrictions	Account Database User Management User Success Management

Class Name: Create Account (Boundary)	
Responsibility:	Collaborators:
Knows User Management Knows Username Knows Password Handles click-event "Create Account"	User Management

Class Name: Account Customization Boundary	
Responsibility:	Collaborators:
Knows User Management Knows Username Knows Password Knows Gender Knows Date-of-birth Knows Dietary Restrictions Handles click-event of "Apply" button	User Management

Class Name: Account Success (Boundary)	
Responsibility:	Collaborators:
Knows User Management Handles unsuccessful events of saving/ creating account	User Management

Class Name: User Success Management (Controller)
--

Responsibility:	Collaborators:
Knows Rating out of 10 Knows Rating Success Knows Rating Error Knows User Information Knows Expert Management	Rating out of 10 Rating Success Rating Error User Information Expert Management

Class Name: Question Management (Controller)	
Responsibility:	Collaborators:
Knows Ask A Question Knows Search Question History Knows Delete Question History Knows Question Error	Ask A Question Search Question History Delete Question History Question Error

Class Name: Expert Management (Controller)	
Responsibility:	Collaborators:
Knows Expert Answer Error Knows Expert 2 (Webscraper) Knows Expert 1 (AI) Knows Expert 3 (Trained Model) Knows Question Database Knows Question Information	Expert Answer Error Expert 2 (Webscraper) Expert 1 (AI) Expert 3 (Trained Model) Question Database Question Information

Class Name: User Management (Controller)	
Responsibility:	Collaborators:
Knows Account Database Knows User Information Knows View Account Knows Login Knows Create Account Knows Account Customization Knows Account Success Knows Account Error Knows Edit Account	Account Database User Information View Account Login Create Account Account Customization Account Success Account Error Edit Account

Division of Labour

Nathan Hum:

- Helped to brainstorm ideas on the architecture design. Suggested both why blackboard and repository could be fitting for our app
- 2: Created the Analysis Class Diagram
- 4: Created CRC cards for the classes: Login, View Account, Ask a question, Question Information, Question Error, expert answer error, Question Management Controller and Expert Management Controller

Christina Zhang:

- Helped brainstorm ideas on architecture design.
- 3.2: Wrote subsystem descriptions
- 4: Created CRC cards for the classes: Rating out of 10, Rating Success, Rating Error, User Success Management, User Management

Yash Patolia

- Section 1.1
- 4: Created CRC cards for the classes: Expert 1, Expert 2, Expert 3, Delete Question History, Account Database, Edit Account
- Compiling the entire document in LaTeX

Justin Kwinecki

- Section 1.2
- Section 1.3
- 4: Created CRC cards for the classes Question Database, Search Question History, User Information, Create Account, Account Customization, Account Success

Muhammad Huzaifah

- Participated in group discussions regarding architecture design of system
- Section 3.1: Explaining non-use of other Architectures
- Section 4: Created an CRC card for Account Error

Jasmine Wang

- Helped brainstorm ideas for architecture, suggested why blackboard is the best
- Created multiple iterations of the System Architecture Diagram
- Section 3.1:
 - Identify and explain the overall architecture
 - Provide the reasoning and justification of the choice